

Trabajo práctico integrador - Consigna primera etapa

1. Consigna

Objetivo del proyecto integrador: desarrollar una solución que permita llevar adelante el control del estado de los vehículos.

Se pide:

- Implementar un modelo cliente servidor.
- Implementar modelo API, desacoplar completamente el frontend del backend.
- Implementar el paradigma de objetos.
- Implementar al menos en la relación entre dos objetos inyección de dependencias.

Objetivo del primer entregable: se pide documentar el plan de trabajo y las decisiones técnicas tomadas para la resolución del mismo. Se deberán entregar diagramas y justificaciones. **NO** se requiere la entrega de código en la primera instancia.

Requerimientos:

- Diagrama de clases del sistema.
- Modelo de datos.
- Detalle de las tecnologías a utilizar.
- Tipo de testeo y módulos que se planifican testear.
- Justificación de la elección de tecnología y consideraciones.

Condiciones de entrega:

Deberá entregarse un documento en formato PDF conteniendo todos los requerimientos expuestos.

Fecha de entrega: semana 8

En el caso que no se pueda realizar la entrega en la fecha estipulada, deberá entregar todo junto en la fecha de entrega del proyecto integrador. Si en esa instancia no alcanza a cumplir con los objetivos, podrá realizar la reentrega en la instancia de recuperación correspondiente.

En el caso que se realicen observaciones o requerir correcciones en el primer entregable, estos deberán entregarse resueltos en la segunda entrega.

Requerimientos:

- Diagrama de clases del sistema.
 - Modelo de datos.
 - Detalle de las tecnologías a utilizar.
 - Tipo de testeo y módulos que se planifican testear.
 - Justificación de la elección de tecnología y consideraciones.
-

2. Modelo de Datos

Vehiculo

- id: UUID
- patente: String (única)
- marca: String
- modelo: String
- año: Number
- estadoActual: Enum("Disponible", "En uso", "En mantenimiento", "Dado de baja")

EstadoVehículo (historial)

- id: UUID
- vehiculoid: UUID (Vehiculo)
- fecha: DateTime
- estado: Enum("Disponible", "En uso", "En mantenimiento", "Dado de baja")
- observaciones: String

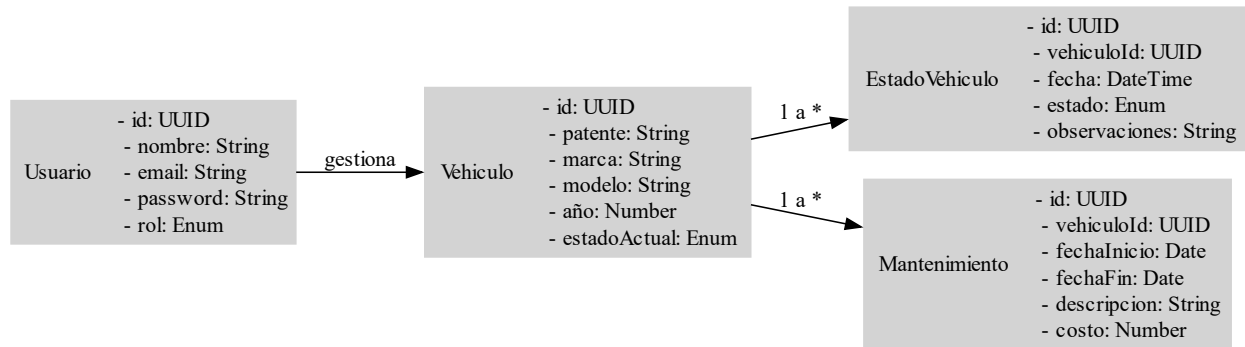
Usuario

- id: UUID
- nombre: String
- email: String
- password: String (hasheado)
- rol: Enum("Administrador", "Operador")

Mantenimiento

- id: UUID
- vehiculoid: UUID (Vehiculo)
- fechaInicio: Date
- fechaFin: Date
- descripcion: String
- costo: Number

3. Diagrama de Clases del Sistema



4. Tecnologías a utilizar

Node.js	Backend escalable para APIs REST.
Express.js	Biblioteca robusta para manejar rutas, middlewares, controladores.
PostgreSQL	Base de datos relacional.
Sequelize (ORM)	Simplifica la persistencia entre objetos y BD.
Jest	Testing unitario e integración en Node.js.
Docker	Facilita despliegue en entornos replicables.
JWT (jsonwebtoken)	Autenticación y autorización segura.

5. Tipo de Testeo y Módulos a Testear

Tipos de Testeo

- **Unitario:** Testear funciones aisladas (validaciones, lógica de datos).
- **Integración:** Testear flujos de datos a través de múltiples componentes (por ejemplo, creación y consulta de funciones).

Módulos a Testear

- Módulo de autenticación de usuarios.
- Alta, modificación y baja lógica de vehículos.
- Cambio de estado de un vehículo y registro en historial.
- Módulo de gestión de mantenimientos.
- Validaciones de fechas (por ejemplo, que un vehículo no pueda estar en uso y mantenimiento al mismo tiempo).

6. Justificación de la Elección Tecnológica y Consideraciones

- **Node.js + Express:** permiten construir un backend ágil y escalable.
- **PostgreSQL:** modelo relacional sólido que asegura consistencia entre vehículos, estados e historial.
- **Sequelize ORM:** mejora productividad evitando SQL repetitivo y mantiene coherencia con el modelo de objetos.
- **Docker:** asegura portabilidad, simplifica instalación en producción y desarrollo.
- **Jest:** asegura calidad del software desde etapas tempranas.
- **JWT:** estándar para sesiones seguras y escalables en aplicaciones modernas.